Week 3 Module:

Date: 29/06/2026

Question 1 (Easy):

A university's Training and Placement Cell have successfully completed its annual placement drive. The placement department has collected the salary packages (in LPA) offered to students by different companies. However, the records were entered in the order they were received, resulting in an unsorted list of package values.

Before publishing the placement statistics report, the department wants all package values arranged in ascending order. Since the university plans to handle placement data for thousands of students in the coming years, they want a sorting technique that is efficient for large datasets.

Your task is to implement the **Merge Sort algorithm** to sort the package values in increasing order. Merge Sort follows a divide-and-conquer approach by dividing the dataset into smaller subarrays, sorting them recursively, and then merging them back together.

The final sorted list will help the university identify minimum, median, and maximum packages while generating analytical reports.

Input

- First line contains an integer **N**, representing the number of placed students.
- Second line contains **N space-separated integers**, representing package values (multiplied by 100 to avoid decimals).

Output

Print the package values in ascending order.

Constraints

- $1 \leq N \leq 100000$
- $100 \leq \text{Package} \leq 10000$

Example

Input:

6
450 1200 800 650 300 1500

Output:

300 450 650 800 1200 1500

Q->1-> Merge sort 3rd year week-3- module.

```
#include <iostream>
#include <vector>
using namespace std;
// function for merging
void merge (vector<int> &arr, int low, int mid, int high) {
    vector<int> temp;
    int left = low;
    int right = mid+1;
    while (left <= mid && right <= high) {
        if (arr[left] < arr[right]) {
        temp.push_back (arr[left]);
        left++;
        }
        else {
            temp.push_back (arr[right]);
            right++;
            }
    }
    while (left<=mid){
        temp.push_back (arr[left]);
        left++;
    }
    while (right<=high) {
        temp.push_back (arr[right]);
        right++;
    }
    for (int i= low; i<=high ; i++) {
        arr[i]= temp[i-low];
    }
}
// function for merge sort.
void mege_sort (vector<int> &arr, int low, int high) {
    if(low>=high) {
        return;
    }
```

```
int mid=(low+high)/2;
merge_sort(arr,low,mid);
merge_sort(arr,mid+1,high);
merge(arr,low,mid,high);
}
int main(){
//input for total number of placed students.
 int n;
 cout<<"Enter the total number of placed students: ";
 cin>>n;
//vector for storing package values.
 vector<int> arr;
 cout<<"Enter the package values: ";
 for(int i=0;i<n;i++){
     int x;
     cin>>x
     arr.push_back(x);
   }
//calling merge sort fn.
   merge_sort(arr,0,n-1);
   cout<<"sorted Package values-> ";
   for(int i=0;i<n;i++){
        cout<<arr[i]<<" ";
    }
   return 0;
}
```

```
PS D:\codez\DS batch\DS CPP module3> cd "d:\codez\DS batch\DS CPP module3\" ; if ($?) { g++ Q1_Merge_sort_easy.cpp -o Q1_Merge_sort_easy } ; if ($?) { .\Q1_Merg
e_sort_easy }
Enter the total number of placed students: 6
Enter the package values: 450
1200
800
650
300
1500
Sorted Package Values -> 300 450 650 800 1200 1500
```

Question 2 (Medium):

A large e-commerce company tracks the time taken (in minutes) to process customer orders before shipment. Due to varying warehouse workloads, processing times differ significantly from one order to another.

The analytics team wants to study operational efficiency by sorting all processing times and generating statistical information. Since millions of orders are processed every month, the company requires an efficient sorting technique with predictable performance.

Your task is to use **Merge Sort** to arrange all processing times in ascending order.
After sorting, perform the following operations:

1. Display the sorted processing times.
2. Find the **median processing time**.
3. Count the number of orders whose processing time is greater than the median.
4. Display the difference between the fastest and slowest processing time.

The solution should efficiently handle large datasets and demonstrate a clear understanding of Merge Sort's divide-and-conquer strategy.

Input

- First line contains integer **N**.
- Second line contains **N space-separated processing times**.

Output

- Sorted processing times.
- Median processing time.
- Count of orders greater than median.
- Difference between maximum and minimum processing time.

Constraints

- $1 \leq N \leq 100000$
- $1 \leq Time \leq 100000$

Example

Input:

7
12 25 18 30 15 22 40

Output:

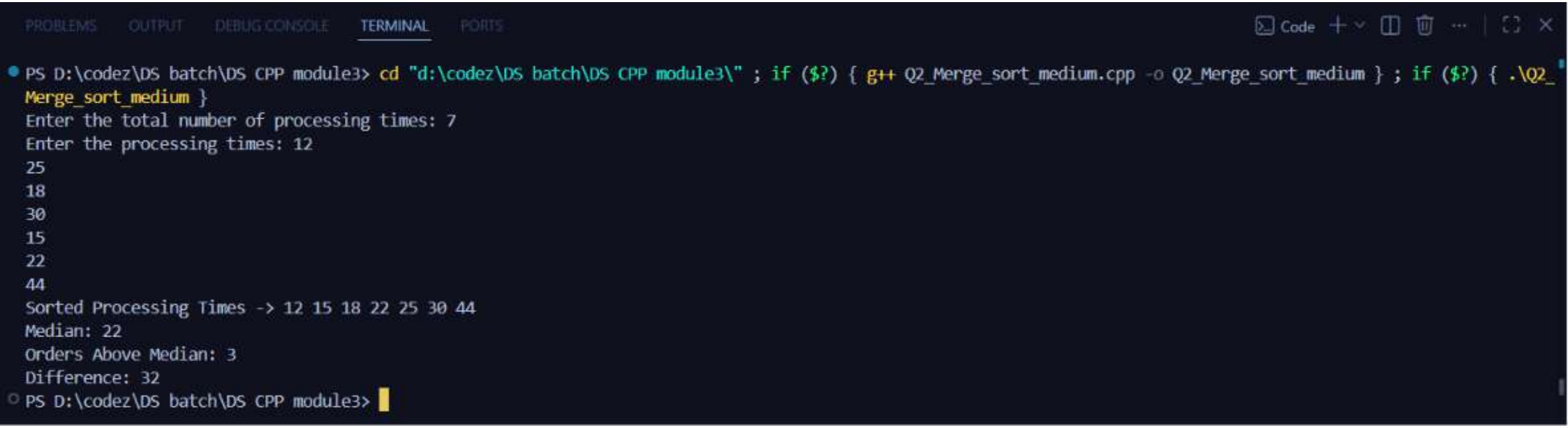
12 15 18 22 25 30 40
Median: 22
Orders Above Median: 3
Difference: 28

Q-2 → Merge_sort-medium.

```
#include<iostream>
#include<vector>
using namespace std;
//function for merging;
void merge(vector<int>&arr, int low, int mid, int high){
    vector<int> temp;
    int left = low;
    int right = mid+1;
    while(left<=mid && right<=high){
     if(arr[left]<arr[right]){
            temp.push_back(arr[left]);
            left++;
      }
      else{
        temp.push_back(arr[right]);
        right++;
      }
    }
   while(left<=mid){
       temp.push_back(arr[right]);
      right++;
    }
  for(int i=low; i<=high; i++){
          arr[i]=temp[i-low];
    }
  }
  //function for merge sort
   void merge_sort(vector<int>&arr, int low, int high){
         if(low>=high){
           return;
         }
       int mid=(low+high)/2;
       merge_sort(arr, low, mid);
       merge_sort(arr, mid+1, high);
```

```
merge(arr, low, mid, high);
}
int main(){
    // input for total number of processing times.
    int n;
    cout<<"Enter the total number processing times: ";
    cin>>n;
    // vector for storing processing times.
    vector<int> arr;
    cout<<"Enter the processing times: ";
    cin>>n;
    for(int i=0; i<n; i++){
        int x;
        cin>>x;
        arr.push_back(x);
    }
    // calling merge sort.
    merge_sort(arr, 0, n-1);
    // printing sorted processing times.
    cout<<"sorted processing Times-> ";
    for(int i=0; i<n; i++){
        cout<<arr[i]<<" ";
    }
    cout<<"\n";
    // finding median
    int median;
    if(n%2==0){
        median=(arr[n/2]+arr[(n/2)-1])/2;
    }
    else{
        median=arr[n/2];
    }
    cout<<"median:"<<median<<endl;
```

```
// counting orders greater than median.
int count=0;
for(int i=0; i<n; i++){
    if(arr[i]> median){
        count++;
    }
}
cout<<" orders Above Median : "<< count <<endl;
// counting orders greater than median.
int count=0
for(int i=0; i<n; i++){
    if(arr[i] >median){
        count++;
    }
}
cout<<" orders Above Median : "<< count <<endl;
// difference b/w maximum & minimum.
int diff = arr[n-1]-arr[0];
cout<< "Difference : "<<diff;
return 0;
}.
```


PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
Code
PS D:\codez\DS batch\DS CPP module3> cd "d:\codez\DS batch\DS CPP module3\" ; if ($?) { g++ Q2_Merge_sort_medium.cpp -o Q2_Merge_sort_medium } ; if ($?) { .\Q2_Merge_sort_medium }
Enter the total number of processing times: 7
Enter the processing times: 12
25
18
30
15
22
44
Sorted Processing Times -> 12 15 18 22 25 30 44
Median: 22
Orders Above Median: 3
Difference: 32
PS D:\codez\DS batch\DS CPP module3>

Topic 2: Quick Sort

Question 1 (Easy):

A cloud infrastructure company manages hundreds of servers distributed across different geographical locations. Every hour, each server reports its average response time in milliseconds. The collected data is stored in the order it arrives and is not sorted.

To generate performance reports, the operations team wants the response times arranged from the fastest server to the slowest server. Since the data size can become very large, an efficient in-place sorting technique is required.

Your task is to implement **Quick Sort** to sort the response times in ascending order. Built-in sorting methods are not allowed.

After sorting, the operations team will use the report to identify high-latency servers and optimize system performance.

Input

- First line contains integer **N**.
- Second line contains **N space-separated response times**.

Output

Print the response times in ascending order.

Constraints

- $1 \leq N \leq 100000$
- $1 \leq$ Response Time ≤ 10000

Example

Input:

6
120 45 78 200 60 95

Output:

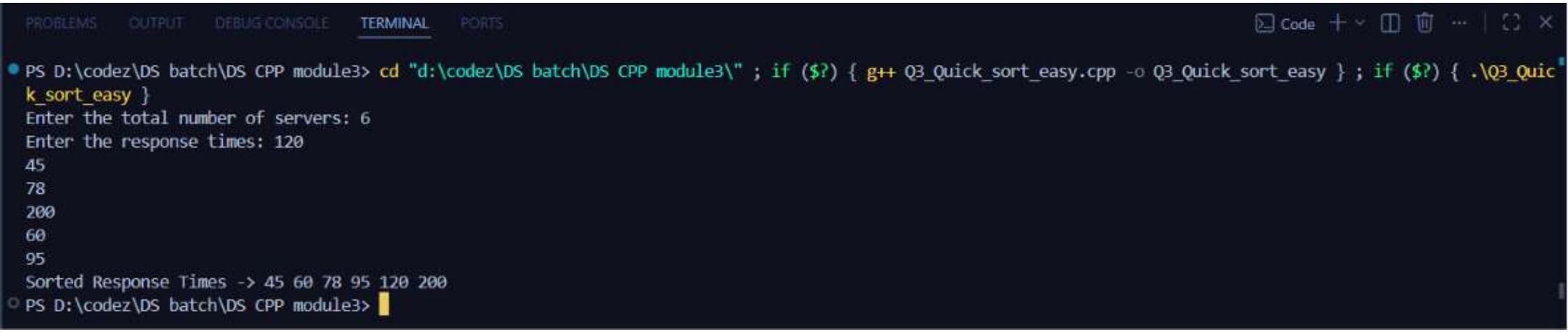
45 60 78 95 120 200

Q-3 -> Quick_sort easy.

```
#include<iostream>
#include<vector>
using namespace std;
//function for partition.
int partition(vector<int>&arr, int low, int high){
    int pivot = arr[low];
    int i = low;
    int j= high;
    while(i<j){
      while(arr[i]<=pivot&&i<high){
          i++;
      }
    while(arr[j]>pivot&&j>low){
          j--;
      }
    if(i<j){
        int t;
        t=arr[i];
       arr[i]=arr[j];
       arr[j]= t;
    }
  }
  int t;
  t= arr[low];
  arr[low]= arr[j];
  arr[j]= t;
  return j;
}

//function for quick sort
void quick_sort(vector<int>&arr. int low, int high){
      if(low<high){
```

```
    int p = partition (arr, low, high);
    quick_sort(arr, low, p-1);
    quick_sort (arr, p+1, high);
    }
};
int main(){
    //input for total number of servers:
    int n;
    cout<<" Enter the total number of servers: ";
    cin>>n;
    //vector for storing response time
    vector<int> arr;
    cout <<" Enter the response times: ";
    for(int i=0 ; i<n ; i++){
        int x;
        cin>>x;
        arr.push_back(x);
    }
    //calling quick sort
    quick_sort(arr, 0, n-1);
    // printing sorted response times
    cout<<"Sorted Response times -> ";
    for(int i=0; i<n; i++){
        cout<<arr[i]<<" ";
    }
    return 0;
};
```


PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
Code
PS D:\codez\DS batch\DS CPP module3> cd "d:\codez\DS batch\DS CPP module3\" ; if ($?) { g++ Q3_Quick_sort_easy.cpp -o Q3_Quick_sort_easy } ; if ($?) { .\Q3_Quick_sort_easy }
Enter the total number of servers: 6
Enter the response times: 120
45
78
200
60
95
Sorted Response Times -> 45 60 78 95 120 200
PS D:\codez\DS batch\DS CPP module3>

Question 2 (Medium):

A financial analytics company receives daily trade-value data from multiple stock exchanges. Analysts use this information to identify high-value trading activity and market trends.

The trade values are received in random order throughout the day. Before analysis can begin, the data must be sorted in descending order.

Your task is to implement **Quick Sort** to arrange the trade values from highest to lowest.

After sorting, perform the following operations:

1. Display the sorted trade values.
2. Find the **Top 5 highest trade values**.
3. Calculate the average of the Top 5 values.
4. Count how many trade values are greater than the overall average of all trade values.

This problem evaluates your understanding of partitioning, recursion, and real-world data analysis using Quick Sort.

Input

- First line contains integer **N**.
- Second line contains **N space-separated trade values**.

Output

- Sorted trade values (descending).
- Top 5 values.
- Average of Top 5 values.
- Count of values greater than overall average.

Constraints

- $5 \leq N \leq 100000$
- $1 \leq$ Trade Value $\leq 10^9$

Example

Input:

8
500 1200 700 2000 900 1500 3000 2500

Output:

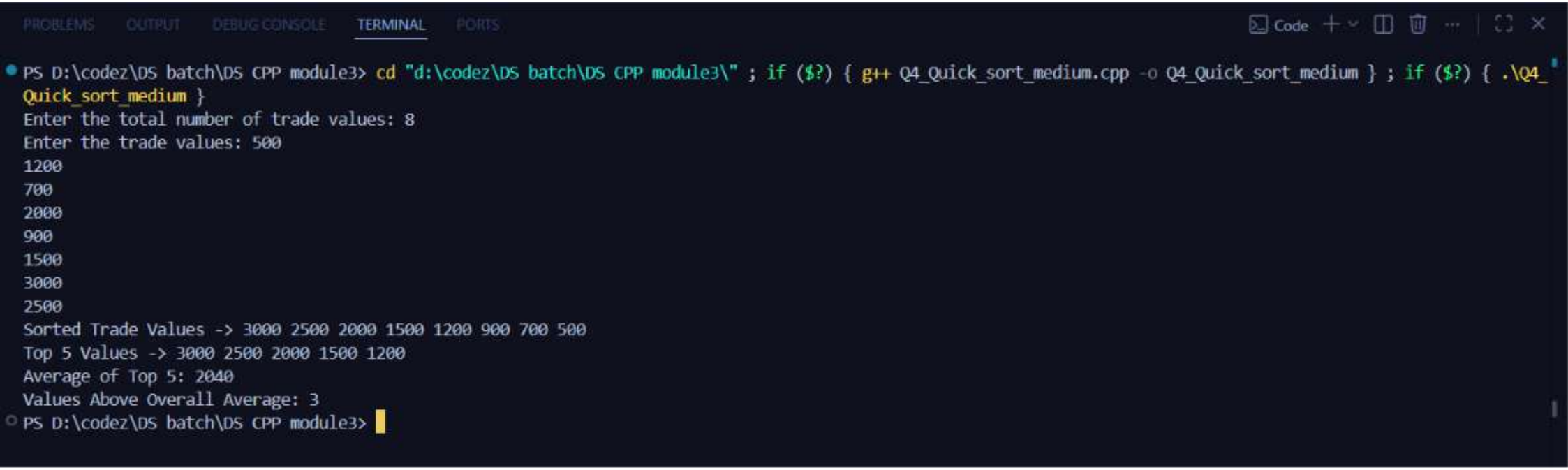
3000 2500 2000 1500 1200 900 700 500
Top 5: 3000 2500 2000 1500 1200
Average of Top 5: 2040
Values Above Overall Average: 4

Q->4 -> Quick sort Medium

```
#include<iostream>
#include<vector>
using namespace std;
// function for partition
int partition(vector<int>&arr, int low, int high){
    int pivot = arr[low];
    int i = low;
    int j = high;
    while(i<j){
        while(arr[i]>=pivot && i<high){
            i++;
        }
        while(arr[j]<pivot && j>low){
            j--;
        }
        if(i<j){
            int t;
            t = arr[i];
            arr[i] = arr[j];
            arr[j] = t;
        }
    }
    int t;
    t = arr[low];
    arr[low] = arr[j];
    arr[j] = t;
    return j;
}
// function for quick sort
void quick_sort(vector<int>&arr, int low, int high){
        if(low<high){
            int p = partition(arr, low, high);
            quick_sort(arr, low, p-1);
            quick_sort(arr, p+1, high);
        }
}
```

```
int main(){
    //input for total number of trade values.
     int n;
     cout<<" Enter the total number of trade values: ";
     cin>>n;
    //vector for storing trade values
     vector<int> arr;
     cout <<"Enter the trade value: ";
     for(int i=0; i<n; i++){
         int x;
        cin>>x;
       arr.push_back(x);
     }
     //calling quick sort
      quick_sort(arr, 0, n-1);
     //printing sorted trade values.
      cout <<"sorted the trade values -> ";
      for(int i=0; i<n; i++){
            cout << arr[i] << "  ";
       }
      cout <<"\n Top 5 values-> ";
      int sum= 0;
      for(int i=0; i<5; i++){
            cout <<arr[i] <<" ";
            sum=sum+arr[i];
        }
    float avg_top5=(float)sum/5;
    cout<<"\n Average of top5 :"<<avg_top5;
    int total=0;
    for(int i=0;i<n;i++){
        total= total+arr[i];
      }
    float overall_avg = (float) total/n;
     int count=0;
     for( int i=0 ; i<n; i++){
        if(arr[i]> overall_avg){
            count++;
        }
      }
```

```
cout<<"\n Values Above Overall Averge: "<<count;
return 0;
}
```


PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
Code
PS D:\codez\DS batch\DS CPP module3> cd "d:\codez\DS batch\DS CPP module3\" ; if ($?) { g++ Q4_Quick_sort_medium.cpp -o Q4_Quick_sort_medium } ; if ($?) { .\Q4_
Quick_sort_medium }
Enter the total number of trade values: 8
Enter the trade values: 500
1200
700
2000
900
1500
3000
2500
Sorted Trade Values -> 3000 2500 2000 1500 1200 900 700 500
Top 5 Values -> 3000 2500 2000 1500 1200
Average of Top 5: 2040
Values Above Overall Average: 3
PS D:\codez\DS batch\DS CPP module3>

Topic 3: Heap Sort

Question 1 (Easy):

An online gaming platform is hosting a national-level tournament involving thousands of participants. At the end of the tournament, player scores are collected from multiple game servers and stored in random order.

Before announcing the final rankings, the organizers need all scores sorted in ascending order. Since the leaderboard data can be very large, they have decided to use **Heap Sort**, which provides guaranteed O(N log N) performance.

Your task is to implement Heap Sort and arrange all player scores in increasing order.

The sorted output will help tournament officials generate rankings and determine qualification thresholds for future events.

Input

- First line contains integer **N**.
- Second line contains **N space-separated player scores**.

Output

Print the scores in ascending order.

Constraints

- $1 \leq N \leq 100000$
- $0 \leq Score \leq 1000000$

Example

Input:

6
850 1200 950 600 1500 1000

Output:

600 850 950 1000 1200 1500

Q->5-> Heap Sort easy.

```
#include <iostream>
#include <vector>
using namespace std;
//function for heapify
void heapify(vector<int>&arr, int n, int i){
    int largest=i;
    int left=(2*i)+1;
    int right=(2*i)+2;
    if(left<n && arr[left]>arr[largest]){
        largest=left;
    }
    if(right<n && arr[right]>arr[largest]){
        largest=right;
    }
    if(largest!=i){
        int t;
        t=arr[i];
        arr[i]=arr[largest];
        arr[largest]=t;
        heapify(arr,n,largest);
    }
}
//function for heap sort
void heap_sort(vector<int>&arr, int n){
    //building Max heap
    for(int i=(n/2)-1; i>=0; i--){
        heapify(arr,n,i);
    }
    //sorting the heap
    for(int i=n-1; i>0; i--){
        int t;
        t=arr[0];
        arr[0]=arr[i];
        arr[i]=t;
        heapify(arr,i,0);
    }
}
```

```
int main() {
    //input for total Number of player scores.
    int n;
    cout<<"Enter the total number of player scores: ";
    cin>>n;
    //vector for storing player score
    vector<int>arr;
    cout<<"Enter the player scores: ";
    for(int i=0; i<n; i++){
        int x;
       cin>>x;
       arr.push_back(x);
    }
    //calling heap sort
     heap_sort(arr,n);
    //printing sorted player scores
     cout<<"Sorted player scores-> ";
     for(int i=0; i<n ;i++){
         cout<<arr[i]<<" ";
        }
     return 0;
}.
```


PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
Code
PS D:\codez\DS batch\DS CPP module3> cd "d:\codez\DS batch\DS CPP module3\" ; if ($?) { g++ Q5_Heap_sort_easy.cpp -o Q5_Heap_sort_easy } ; if ($?) { .\Q5_Heap_sort_easy }
Enter the total number of player scores: 6
Enter the player scores: 850
1200
950
600
1500
1000
Sorted Player Scores -> 600 850 950 1000 1200 1500
PS D:\codez\DS batch\DS CPP module3>

Question 2 (Medium):

A multi-specialty hospital maintains records of emergency response times for all critical cases handled during a month. Each response time represents the number of minutes taken by the emergency team to reach and begin treatment for a patient.

Hospital administrators want to analyze the efficiency of their emergency services. To do so, all response times must first be sorted using **Heap Sort**.

After sorting the response times in ascending order, generate the following statistics:

1. Display the sorted response times.
2. Find the fastest response time.
3. Find the slowest response time.
4. Calculate the average response time.
5. Count the number of cases handled faster than the average.
6. Calculate the percentage of cases handled faster than the average.

The hospital intends to use this information to improve resource allocation and reduce emergency response delays.

Input

- First line contains integer **N**.
- Second line contains **N space-separated response times**.

Output

- Sorted response times.
- Fastest response time.
- Slowest response time.
- Average response time.
- Number of cases faster than average.
- Percentage of cases faster than average.

Constraints

- $1 \leq N \leq 100000$
- $1 \leq Time \leq 500$

Example

Input:

8
12 7 15 9 20 5 10 8

Output:

5 7 8 9 10 12 15 20
Fastest: 5
Slowest: 20
Average: 10.75
Cases Faster Than Average: 5
Percentage: 62.50%

Q-16) Heap sort.

```
#include <iostream>
#include <vector>
using namespace std;
// function for heapify.
void heapify(vector<int>&arr, int n, int i){
    int largest=i;
    int left=(2*i)+1;
    int right=(2*i)+2;
if(left<n && arr[left]>arr[largest]){
        largest=left;
    }
if(right<n && arr[right]>arr[largest]){
        largest=right;
    }
if(largest!=i){
    int t;
    t=arr[i];
    arr[i]=arr[largest];
    arr[largest]=t;
    heapify(arr,n,largest);
    }
}
// function for heap sort
void heap_sort(vector<int>&arr, int n){
        // building max heap.
        for(int i=(n/2)-1; i>=0; i--){
            heapify(arr,n,i);
        }
        // sorting the heap.
```

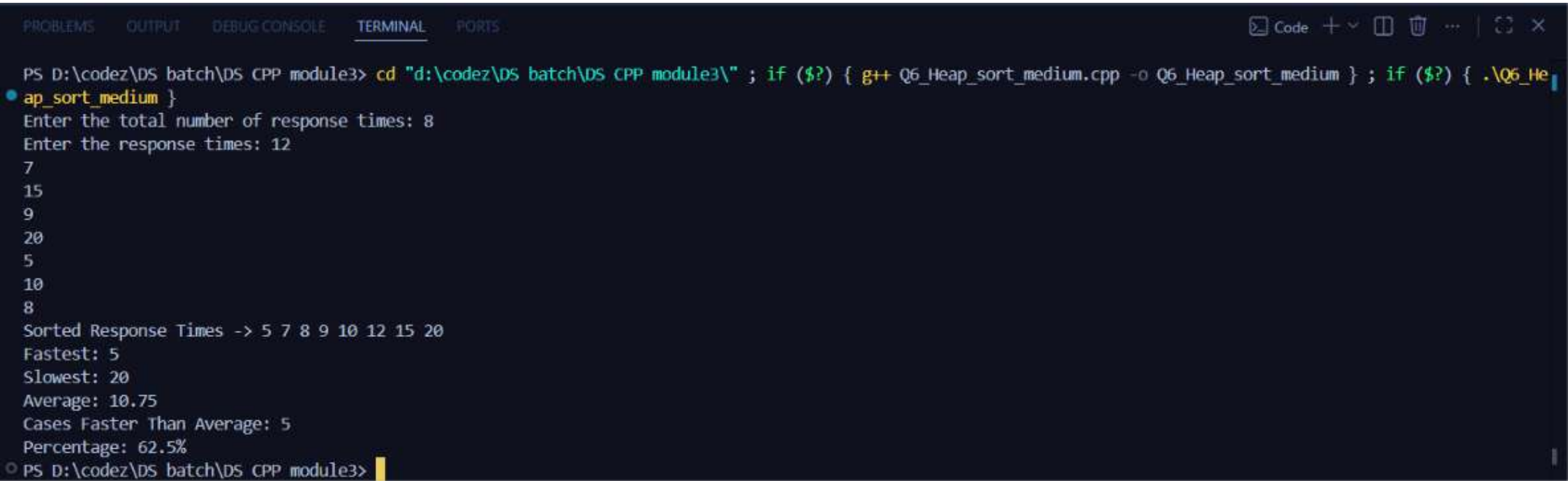
```
for(int i=n-1 ; i> 0 ; i--){
        int t;
        t=arr[0];
        arr[0]=arr[i];
        arr[i]= t;
        heapify(arr,i,0);
    }
}

int main(){
    // input for totalnumber of response times.
     int n;
     cout<<"Enter the totalnumber of response times: ";
     cin>>n;

     //vector for ~~sort~~ storing response time
     vector<int> arr;
     cout<<"Enter the response time: ";
     for(int i=0;i<n;i++){
         int x;
         cin>>x;
         arr.push_back(x);
      }
    // calling heap sort
     heap_sort(arr,n);
    // printing sorted response time
    cout<<"Sorted Response times-> ";
    for(int i=0;i<n;i++){
        cout<<arr[i]<<" ";
     }
    cout<<"\n Fastest: "<<arr[0];
    cout<<"\n Slowest: "<<arr[n-1];
```

```
int sum=0;
for(int i=0;i<n; i++){
        sum=sum+arr[i];
   }.

float avg=(float)sum/n;
 cout<<"\n Average : "<<avg;
 int count=0;
 for(int i=0 ;i<n;i++){
        if(arr[i]<avg){
            count++;
        }
   }
   cout<<"\n Cases Faster than Average :"<<count;
   float per=((float)count/n)*100;
   cout<<"\n percentage : "<<per<<"%";
   return 0;
}.
```


PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
Code
PS D:\codez\DS batch\DS CPP module3> cd "d:\codez\DS batch\DS CPP module3\" ; if ($?) { g++ Q6_Heap_sort_medium.cpp -o Q6_Heap_sort_medium } ; if ($?) { .\Q6_Heap_sort_medium }
Enter the total number of response times: 8
Enter the response times: 12
7
15
9
20
5
10
8
Sorted Response Times -> 5 7 8 9 10 12 15 20
Fastest: 5
Slowest: 20
Average: 10.75
Cases Faster Than Average: 5
Percentage: 62.5%
PS D:\codez\DS batch\DS CPP module3>